VITYARTHI — Open Source Software Course | 2025–26

The Open Source Audit

A Capstone Project for OSS NGMC Course

Student Name	Yash Tripathi
Registration Number	24BCE10603
Slot	B22
Date of Submission	29 March 2026
Chosen Software	Git — Version Control System
Licence	*GNU GPL v2*

Part A — Origin and Philosophy

A1. The Problem Git Was Created to Solve

How It Actually Started

People often say Linus simply needed a version control system, but that skips the interesting part. In 2002, the Linux kernel team began using BitKeeper, a proprietary tool from BitMover that they could use for free—but on the company's terms. It wasn't ideal, but there were few alternatives.

Then in April 2005, Andrew Tridgell reverse-engineered BitKeeper's protocol to create an open alternative. BitMover revoked the Linux project's access, leaving thousands of developers without a VCS. With the next kernel merge window approaching, Linus reviewed existing tools. He disliked CVS, and Subversion, while better, lacked the performance and distributed model they needed.

So he built his own. In about ten days, the first version of Git was ready, and the kernel project switched to it within weeks.

Why 'Distributed' Was the Point

Before Git, systems like CVS and Subversion used a client-server model with a single central server. If the server went down or was slow, work stopped. The server owner also controlled the project.

Git gave every developer a complete copy of the repository and full history. Work could continue offline, and there was no single point of failure. This design was a direct response to the BitKeeper incident: it ensured no central authority could ever halt the project again.

What Git Actually Changed

I feel like people often underestimate how different things were before Git. In tools like Subversion, branching wasn't something you could just do casually — it took time and added complexity, so teams usually avoided it. Most of the work happened on a single main branch, and developers had to coordinate carefully to avoid conflicts. Merging changes was often messy and could easily go wrong. The idea of having lots of parallel branches, with people working independently and merging whenever they wanted, just wasn't really practical back then.

Git changed that completely. Creating a branch takes almost no time, and merging is much smoother. Because of that, the way people collaborate — especially in open source — shifted a lot. GitHub, which came later, is basically built around this idea of easy branching. Even the pull request workflow that most projects use today depends on the fact that branching is so lightweight.

Sometimes when I run git log and see a long history of contributions from so many different people, all organised and easy to track, it's kind of surprising. It all traces back to the design decisions Torvalds made in a very short time back in 2005. It's a bit strange to think that something built so quickly ended up shaping how almost all software is developed today.

A2. The Licence — What It Actually Says

The Four Freedoms — and Whether Git Actually Delivers Them
The Free Software Foundation defines four freedoms that a piece of software must grant for it to be considered 'free software':
Freedom 0 — the freedom to run the program for any purpose
Freedom 1 — the freedom to study the source code and change it
Freedom 2 — the freedom to redistribute copies
Freedom 3 — the freedom to distribute your modified versions
Git's Licence: GPL v2
Git is licensed under the GNU General Public Licence version 2 (GPL v2), which grants four core freedoms: you can download, read, modify, and ship your own version. A Fortune 500 company and a student have exactly the same rights. There are no restrictions on what you use it for.
Can a Company Modify and Sell Git Without Sharing?
No. This is where the GPL differs from permissive licences like MIT or Apache. The GPL includes a *copyleft* clause: if you modify Git and distribute that version to others, you must release your source code under the same GPL v2 licence. You cannot turn it into a closed, proprietary product. The freedom must continue down the line to every recipient. This prevents community work from being closed off later.
GPL vs MIT — Which Would I Choose?
MIT: If building a library for other developers, I'd choose MIT. It removes friction and makes adoption easier, since companies are often cautious about GPL's legal obligations.
GPL: If building important infrastructure that I don't want taken and closed off, I'd choose GPL. Projects like Git and the Linux kernel made this choice to ensure the code always remains open.
There is no single correct licence. MIT prioritises wide adoption; GPL preserves ongoing freedom.
Free as in Free Beer vs Free as in Freedom
Free beer: costs no money.

Freedom (libre): you can see the source code, modify it, and are not locked in.
A company can offer a free tool, but if you cannot access the code, modify it, or leave without losing your data, you lack control. Git is free in both senses, but the freedom part is what truly matters—and it was created to avoid the lock-in situation that BitKeeper caused.

A3. The Ethics of Open Source

Should All Software Be Open Source?
No, but critical infrastructure should be. Tools like Git, Linux, and OpenSSL underpin the internet and shouldn't be controlled by a single company. Openness enables transparency and rapid response to issues like Heartbleed. For games or creative tools, commercial incentives matter, and I use proprietary software myself. But infrastructure demands a higher standard.
Is It Ethical to Profit from Community Labour?
The licence allows it, but that's not a complete answer. Many tech companies, like AWS, built fortunes on open source without always giving back proportionally. There's a difference between active contributors (e.g., Red Hat) and those who simply consume. While not a legal requirement, companies profiting significantly from shared infrastructure have a strong ethical obligation to contribute back.
Standing on the Shoulders of Giants
All software builds on prior work. Git combined ideas from Monotone and BitKeeper, SHA-1, and Unix philosophy. This is why open source advances faster collectively—shared knowledge lets everyone improve together, not just individual companies moving alone.

Part B — Linux Footprint

Installation

Git is available through all major Linux package managers. On a Debian/Ubuntu system the command is 'sudo apt install git'. On Fedora or RHEL it's 'sudo dnf install git'. On Arch it's 'sudo pacman -S git'. For those who want the very latest version or need custom compile options, building from source is also possible — download the tarball from git-scm.com and run ./configure, make, then sudo make install.

The package-manager route is what I'd recommend for most cases. It ties Git into the system's update mechanism, which means security patches land automatically alongside the rest of your system updates. That's not a small thing — Git has had security vulnerabilities, and prompt patching matters.

Key Directories

After installation, Git distributes itself across several directories following the Linux Filesystem Hierarchy Standard:

Directory	Contents	What it's for
/usr/bin/git	Main binary	The git command you actually run
/usr/lib/git-core/	Sub-commands	git-pull, git-push, git-fetch etc.
/usr/share/git-core/	Templates	Default hooks and template files
/etc/gitconfig	System config	Settings that apply to every user
~/.gitconfig	User config	Your personal name, email, aliases
.git/ (per repo)	Repository data	All commits, history, branches, refs

User, Group, and Security

For local usage, Git runs as whatever user is currently logged in. There's no dedicated system user for Git in that context. On a server hosting shared repositories, however, a dedicated 'git' user account is typically created, and that user is given a restricted shell called git-shell. This means that even if someone gains SSH access as the git user, they can only interact with repositories — they cannot run arbitrary commands on the server.

Repository directories on a shared server are owned by git:git, usually with permissions 755 or sometimes 750 depending on whether repositories should be readable by other local users. This follows the principle of least privilege — the git account has exactly the access it needs and nothing more.

Service Management

Git is not a daemon — it doesn't run continuously in the background during normal usage. But when used as a server, it's commonly paired with services that do. The three main options:

- git daemon — a lightweight read-only server for the Git protocol. Managed with: sudo systemctl start git-daemon.socket
- SSH — the most common way to host Git remotely. You're actually managing sshd, not Git directly: sudo systemctl status sshd
- Gitea or GitLab — full web-based hosting platforms, both managed as systemd services: sudo systemctl start gitea

Part C — The FOSS Ecosystem

What Git Depends On

For something so widely used, Git's dependencies are surprisingly lean. This is partly why it runs on everything from embedded systems to mainframes.

- OpenSSL or GnuTLS — handles the cryptographic side of HTTPS connections when you push or pull over HTTPS.
- libcurl — manages the actual HTTP/HTTPS communication for Git's smart HTTP protocol.
- zlib — compresses objects before storing or transmitting them. This is why Git repositories are compact relative to the actual file sizes they track.
- glibc — the standard C library. Git's core is written in C and relies on it for everything from file I/O to memory management.
- Perl — some of the older Git scripts and the git-svn bridge are written in Perl.

What Git Has Made Possible

If I had to answer honestly, I'd say a huge part of modern software development depends on Git in some way. GitHub, which started in 2008, now hosts hundreds of millions of repositories. GitLab came along as an alternative, and the fact that you can self-host it is a big deal for organisations that can't rely on external servers. Both platforms are built entirely around Git, so without it, they wouldn't really exist in the same form.

Tools like GitHub Actions and GitLab CI also show how much Git has shaped workflows. These systems automatically run tests or deploy code whenever someone pushes a commit. That kind of automation has become normal now, but it wasn't always like this. Continuous integration existed before, but Git made it feel like a default part of development rather than something extra.

More recently, platforms like Hugging Face have started using Git as well, especially with Git LFS for handling large files like machine learning models. Git wasn't originally designed with machine learning in mind, but it turns out the core idea was flexible enough to support it anyway. It's interesting how something built for one purpose ended up being useful in areas that didn't even exist at the time.

Community and Governance

Git's development happens primarily on a public mailing list — git@vger.kernel.org — where patches are submitted, reviewed, and discussed. Junio C. Hamano has been the primary maintainer since Torvalds handed stewardship over in 2005. He's not employed by any single company — he works at Google, but maintains Git independently of that.

Microsoft's 2018 acquisition of GitHub raised eyebrows, but Git itself was unaffected — Microsoft doesn't own Git, they own a web platform built on top of it. They have since become significant contributors to the Git project, particularly improving Windows compatibility and performance for very large repositories.

Part D — Open Source vs Proprietary

Git vs Perforce Helix Core

Perforce is the most widely used proprietary version control system, particularly in game development and large enterprises. It's worth comparing honestly because 'open source is always better' is not actually a defensible position — there are real trade-offs.

Dimension	Git (Open Source)	Perforce Helix Core (Proprietary)
Cost	Free — zero licence fees, forever	Per-user licensing; expensive at scale
Code auditability	Full source available at kernel.org	Closed source — trust the vendor
Support	Community + paid support via GitHub, GitLab etc.	Vendor SLA; dedicated enterprise support team
Large binary files	Needs Git LFS extension; manageable but not seamless	Native and strong — built for game/media workflows
Freedom to modify	Complete — fork, patch, redistribute under GPL	None — source is not available
Ecosystem	Enormous — GitHub, GitLab, every CI/CD tool, every IDE	Smaller; mainly enterprise game studios
Community control	No single owner; no vendor lock-in possible	Corporate roadmap; licence can change at any time
Overall verdict	Right choice for virtually all software development	Justified only for very large game/media asset pipelines

My Verdict

If I'm being honest, for most software development, it's hard to argue against Git. The ecosystem around it is just too strong — almost every tool integrates with Git, and pretty much every developer already knows how to use it. The whole collaboration model we see on platforms like GitHub and GitLab, especially with branching and pull requests, really depends on how Git works. There isn't a direct equivalent to that in the Perforce world.

That said, Perforce does have a clear advantage in one area — handling large binary files. In industries like AAA game development, where teams deal with massive assets like textures, 3D models, and audio files, Perforce tends to work better. Git wasn't really designed for that kind of workload. Git LFS helps, but it still doesn't feel as seamless as Perforce's built-in approach. So in that specific scenario, I can see why a team might choose Perforce.

Part E — Shell Script Documentation

Script 1 — System Identity Report

Explanation:

This script displays a welcome screen with information about the Linux system — distribution name, kernel version, current user, home directory, uptime, date and time, and a message about the GPL v2 licence covering both Linux and Git.

Concepts demonstrated

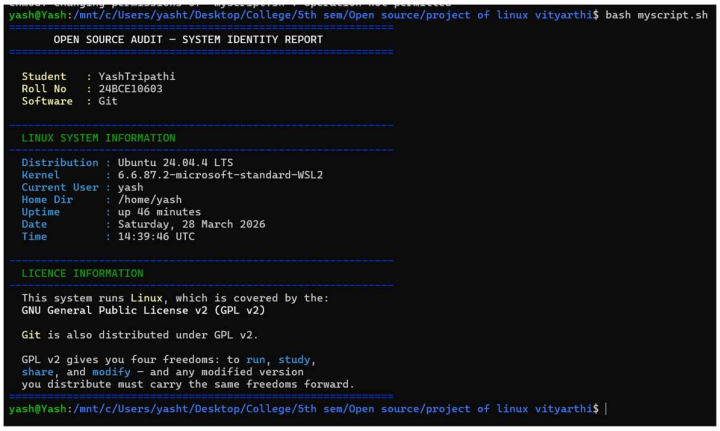
- Variables — STUDENT_NAME, ROLL_NUMBER, SOFTWARE_CHOICE store values used across the script.
- Command substitution — $(uname -r), $(whoami), $(uptime -p), $(date) run commands and capture their output into variables at runtime.
- Output formatting — echo statements with consistent spacing and separator lines produce readable display.
- cat /etc/os-release with grep and cut extracts the human-readable distribution name.

Code:

```
#!/bin/bash
# ============================================================
# Script 1: System Identity Report
# Course  : Open Source Software — VITyarthi
# Author  : Yash Tripathi | Roll No: 24BCE10603
# What it does: Prints a welcome screen showing the Linux
#               system identity and the licence covering it.
# ============================================================
```

```
# ANSI colour codes — makes output readable at a glance
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
CYAN='\033[0;36m'
BOLD='\033[1m'
NC='\033[0m'    # No Colour — always reset after each coloured line
# Student details — fill these in
STUDENT_NAME=YashTripathi
ROLL_NUMBER=24BCE10603
SOFTWARE_CHOICE="Git"
SOFTWARE_LICENSE="GNU General Public License v2 (GPL v2)"
# Gather live system info using command substitution — $(command)
# runs the command and places its output directly into the variable
KERNEL=$(uname -r)
DISTRO=$(grep -w "PRETTY_NAME" /etc/os-release 2>/dev/null | cut -d= -f2 | tr -d '"')
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date '+%A, %d %B %Y')
CURRENT_TIME=$(date '+%H:%M:%S %Z')
# Print the header banner
echo -e "${BLUE}==============================================================${NC}"
echo -e "${BOLD}       OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT          ${NC}"
echo -e "${BLUE}==============================================================${NC}"
echo ""
echo -e "  ${YELLOW}Student   :${NC} $STUDENT_NAME"
echo -e "  ${YELLOW}Roll No   :${NC} $ROLL_NUMBER"
echo -e "  ${YELLOW}Software  :${NC} $SOFTWARE_CHOICE"
echo ""
echo -e "${BLUE}--------------------------------------------------------------${NC}"
# Linux system section
echo -e "  ${GREEN}LINUX SYSTEM INFORMATION${NC}"
echo -e "${BLUE}--------------------------------------------------------------${NC}"
echo -e "  ${CYAN}Distribution :${NC} $DISTRO"
echo -e "  ${CYAN}Kernel       :${NC} $KERNEL"
echo -e "  ${CYAN}Current User :${NC} $USER_NAME"
echo -e "  ${CYAN}Home Dir     :${NC} $HOME_DIR"
echo -e "  ${CYAN}Uptime       :${NC} $UPTIME"
echo -e "  ${CYAN}Date         :${NC} $CURRENT_DATE"
echo -e "  ${CYAN}Time         :${NC} $CURRENT_TIME"
echo ""
echo -e "${BLUE}--------------------------------------------------------------${NC}"
# Licence section
echo -e "  ${GREEN}LICENCE INFORMATION${NC}"
echo -e "${BLUE}--------------------------------------------------------------${NC}"
echo -e "  This system runs ${YELLOW}Linux${NC}, which is covered by the:"
echo -e "  ${BOLD}$SOFTWARE_LICENSE${NC}"
echo ""
echo -e "  ${YELLOW}$SOFTWARE_CHOICE${NC} is also distributed under GPL v2."
echo ""
echo -e "  GPL v2 gives you four freedoms: to ${CYAN}run${NC}, ${CYAN}study${NC},"
```

```
echo -e "  ${CYAN}share${NC}, and ${CYAN}modify${NC} — and any modified version"
echo -e "  you distribute must carry the same freedoms forward."
echo -e "${BLUE}============================================================${NC}"
```



Script 2 — FOSS Package Inspector

Explanation:

This script checks whether Git is installed, handles both RPM-based and Debian-based package managers, prints version and licence metadata, and uses a case statement to display a philosophy note about whichever open-source package is named.

Concepts demonstrated

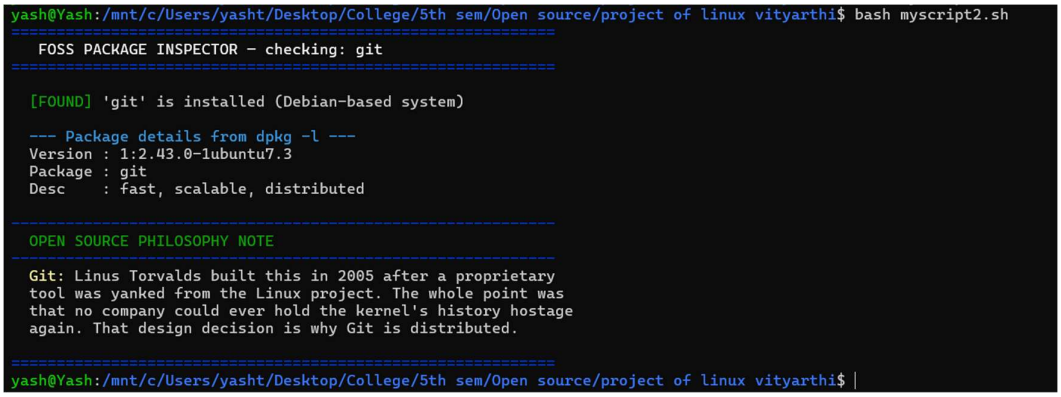
- if-then-else — tests whether rpm or dpkg is available using command -v, then queries the appropriate one.
- case statement — matches the package name against known open-source tools and prints a relevant note for each.
- Pipe with grep — rpm -qi $PACKAGE | grep -E filters package metadata to show only the relevant fields.
- &>/dev/null — suppresses both stdout and stderr when silently testing whether a command succeeds.

Code:

```
#!/bin/bash
# ============================================================
# Script 2: FOSS Package Inspector
# Course  : Open Source Software — VITyarthi
# Author  :Yash Tripathi | Roll No: 24BCE10603
# What it does: Checks if Git is installed, shows its details,
#               and prints a philosophy note via a case statement.
# ============================================================
# ANSI colours
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
CYAN='\033[0;36m'
BOLD='\033[1m'
NC='\033[0m'
# The package we're inspecting — our chosen open-source software
PACKAGE="git"
echo -e "${BLUE}============================================================${NC}"
echo -e "${BOLD}   FOSS PACKAGE INSPECTOR — checking: $PACKAGE            ${NC}"
echo -e "${BLUE}============================================================${NC}"
echo ""
# if-then-else block: detect which package manager is present,
# then query it. &>/dev/null silences both stdout and stderr
# so failed checks don't print errors to the screen.
if command -v rpm &>/dev/null && rpm -q "$PACKAGE" &>/dev/null; then
   # RPM-based system (Fedora, RHEL, CentOS)
```

```
  echo -e "  ${GREEN}[FOUND]${NC} '$PACKAGE' is installed (RPM-based system)"
  echo ""
  echo -e "  ${CYAN}--- Package details from rpm -qi ---${NC}"
  rpm -qi "$PACKAGE" | grep -E 'Version|License|Summary|URL'
elif command -v dpkg &>/dev/null && dpkg -l "$PACKAGE" 2>/dev/null | grep -q "^ii"; then
  # Debian/Ubuntu-based system
  echo -e "  ${GREEN}[FOUND]${NC} '$PACKAGE' is installed (Debian-based system)"
  echo ""
  echo -e "  ${CYAN}--- Package details from dpkg -l ---${NC}"
  dpkg -l "$PACKAGE" | grep "^ii" | \
    awk '{print "  Version : "$3"\n  Package : "$2"\n  Desc    : "$5" "$6" "$7}'
elif command -v git &>/dev/null; then
  # Git present but not via a package manager (compiled from source)
  echo -e "  ${GREEN}[FOUND]${NC} '$PACKAGE' is installed (compiled from source)"
  echo ""
  echo -e "  ${CYAN}--- Version ---${NC}"
  git --version
else
  # Not found at all
  echo -e "  ${RED}[NOT FOUND]${NC} '$PACKAGE' is not installed on this system."
  echo ""
  echo -e "  To install Git:"
  echo -e "    ${YELLOW}Debian/Ubuntu :${NC} sudo apt install git"
  echo -e "    ${YELLOW}Fedora/RHEL   :${NC} sudo dnf install git"
  echo -e "    ${YELLOW}Arch Linux    :${NC} sudo pacman -S git"
fi
echo ""
echo -e "${BLUE}------------------------------------------------------------${NC}"
# case statement: prints an open-source philosophy note
# based on the package name. Each pattern matches a known FOSS tool.
echo -e "  ${GREEN}OPEN SOURCE PHILOSOPHY NOTE${NC}"
echo -e "${BLUE}------------------------------------------------------------${NC}"
case $PACKAGE in
  git)
    echo -e "  ${YELLOW}Git:${NC} Linus Torvalds built this in 2005 after a proprietary"
    echo -e "  tool was yanked from the Linux project. The whole point was"
    echo -e "  that no company could ever hold the kernel's history hostage"
    echo -e "  again. That design decision is why Git is distributed."
    ;;
  httpd|apache2)
    echo -e "  ${YELLOW}Apache:${NC} The web server that proved community-built software"
    echo -e "  could power the internet. At its peak it served over 60%"
    echo -e "  of all websites. Built openly, maintained collectively."
    ;;
  mysql|mariadb)
    echo -e "  ${YELLOW}MySQL:${NC} Its acquisition by Oracle sparked the MariaDB fork —"
    echo -e "  a reminder that even open-source projects can be threatened"
    echo -e "  by corporate ownership. The community voted with their feet."
    ;;
  python3|python)
    echo -e "  ${YELLOW}Python:${NC} Governed by a community foundation, not a company."
    echo -e "  The PSF licence is permissive enough that Python is now"
    echo -e "  the world's most used language. Openness drove adoption."
    ;;
  vlc)
    echo -e "  ${YELLOW}VLC:${NC} Started by students at a French university who wanted"
    echo -e "  to stream video over their campus network. No commercial"
```

```
    echo -e "  tool was open enough to adapt, so they built their own."
    ;;
  firefox)
    echo -e "  ${YELLOW}Firefox:${NC} Mozilla's answer to Internet Explorer's monopoly."
    echo -e "  A nonprofit building a browser to keep the web open."
    echo -e "  Without Firefox, the web would look very different today."
    ;;
  *)
    echo -e "  ${YELLOW}$PACKAGE:${NC} An open-source tool built on the belief that"
    echo -e "  shared knowledge moves faster than proprietary knowledge."
    ;;
esac
echo ""
echo -e "${BLUE}============================================================${NC}"
```



Script 3 — Disk and Permission Auditor

Explanation:

This script loops through a predefined array of important system directories and reports the permissions, ownership, and disk usage of each. It then separately audits Git-specific configuration file locations on the system.

Concepts demonstrated

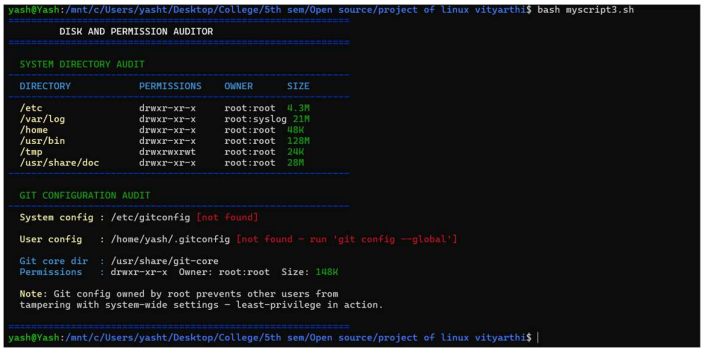
- for loop — iterates over the DIRS array using the ${DIRS[@]} expansion syntax.
- Array declaration — DIRS="("/etc" "/var/log" ...) holds multiple paths for the loop to process.
- ls -ld with awk — extracts the permissions string and owner:group from the long directory listing.
- du -sh with cut — produces a human-readable size figure by cutting the tab-separated du output.
- [-d] test — checks that each path is a real directory before trying to read it.

Code:

```
#!/bin/bash
# ============================================================
# Script 3: Disk and Permission Auditor
# Course  : Open Source Software — VITyarthi
# Author  : Yash Tripathi | Roll No: 24BCE10603
# What it does: Loops through key system directories and reports
#               permissions, ownership, and size for each.
#               Also checks Git's own config file locations.
# ============================================================
# ANSI colours
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
CYAN='\033[0;36m'
BOLD='\033[1m'
```

```
NC='\033[0m'
# Array of important directories to audit
# These are standard locations on any Linux system
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/share/doc")
echo -e "${BLUE}============================================================${NC}"
echo -e "${BOLD}        DISK AND PERMISSION AUDITOR                    ${NC}"
echo -e "${BLUE}============================================================${NC}"
echo ""
echo -e "  ${GREEN}SYSTEM DIRECTORY AUDIT${NC}"
echo -e "${BLUE}------------------------------------------------------------${NC}"
printf "  ${CYAN}%-20s %-14s %-10s %-8s${NC}\n" "DIRECTORY" "PERMISSIONS" "OWNER" "SIZE"
echo -e "${BLUE}------------------------------------------------------------${NC}"
# for loop — iterates over every entry in the DIRS array
# ${DIRS[@]} expands the full array; quotes handle paths with spaces
for DIR in "${DIRS[@]}"; do
   # [ -d "$DIR" ] tests whether the path exists and is a directory
   if [ -d "$DIR" ]; then
      # awk pulls out the permissions string ($1) and owner:group ($3:$4)
      PERMS=$(ls -ld "$DIR" | awk '{print $1}')
      OWNER=$(ls -ld "$DIR" | awk '{print $3":"$4}')
      # du -sh gives human-readable size; cut -f1 takes just the number
      SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
      printf "  ${YELLOW}%-20s${NC} %-14s %-10s ${GREEN}%-8s${NC}\n" \
          "$DIR" "$PERMS" "$OWNER" "$SIZE"
   else
      # Directory doesn't exist on this particular system
      printf "  ${RED}%-20s${NC} %s\n" "$DIR" "[not found on this system]"
   fi
done
echo -e "${BLUE}------------------------------------------------------------${NC}"
echo ""
# Git-specific config directory audit
# This section checks the three tiers of Git configuration on Linux
echo -e "  ${GREEN}GIT CONFIGURATION AUDIT${NC}"
echo -e "${BLUE}------------------------------------------------------------${NC}"
GIT_SYSTEM="/etc/gitconfig"
GIT_USER="$HOME/.gitconfig"
GIT_CORE="/usr/share/git-core"
# Check system-wide git config
if [ -f "$GIT_SYSTEM" ]; then
   PERMS=$(ls -l "$GIT_SYSTEM" | awk '{print $1}')
   OWNER=$(ls -l "$GIT_SYSTEM" | awk '{print $3":"$4}')
   echo -e "  ${CYAN}System config :${NC} $GIT_SYSTEM"
   echo -e "  ${CYAN}Permissions   :${NC} $PERMS  Owner: $OWNER"
else
   echo -e "  ${YELLOW}System config :${NC} $GIT_SYSTEM ${RED}[not found]${NC}"
fi
echo ""
# Check user-level git config
if [ -f "$GIT_USER" ]; then
   PERMS=$(ls -l "$GIT_USER" | awk '{print $1}')
   OWNER=$(ls -l "$GIT_USER" | awk '{print $3":"$4}')
   echo -e "  ${CYAN}User config   :${NC} $GIT_USER"
   echo -e "  ${CYAN}Permissions   :${NC} $PERMS  Owner: $OWNER"
else
   echo -e "  ${YELLOW}User config   :${NC} $GIT_USER ${RED}[not found — run 'git config --global']${NC}"
fi
```

```
echo ""
# Check Git's shared template/hooks directory
if [ -d "$GIT_CORE" ]; then
   SIZE=$(du -sh "$GIT_CORE" 2>/dev/null | cut -f1)
   PERMS=$(ls -ld "$GIT_CORE" | awk '{print $1}')
   OWNER=$(ls -ld "$GIT_CORE" | awk '{print $3":"$4}')
   echo -e "  ${CYAN}Git core dir  :${NC} $GIT_CORE"
   echo -e "  ${CYAN}Permissions   :${NC} $PERMS  Owner: $OWNER  Size: ${GREEN}$SIZE${NC}"
   echo ""
   echo -e "  ${YELLOW}Note:${NC} Git config owned by root prevents other users from"
   echo -e "  tampering with system-wide settings — least-privilege in action."
else
   echo -e "  ${YELLOW}Git core dir  :${NC} $GIT_CORE ${RED}[not found]${NC}"
fi
echo ""
echo -e "${BLUE}============================================================${NC}"
```



Script 4 — Log File Analyzer

Explanation:

This script takes a log file path and an optional keyword as command-line arguments, scans the file line by line counting matches, implements a retry loop for empty files, and prints a summary with the last five matching lines.

Concepts demonstrated

- Command-line arguments — $1 and $2 take the log file and keyword; ${2:-"error"} provides a default value if none is given.
- while IFS= read -r — reads the log file line by line, with IFS= and -r preserving whitespace and backslashes correctly.
- Counter variable — COUNT=$(($COUNT + 1)) uses arithmetic expansion to track keyword occurrences.
- Do-while simulation — a while true loop with break implements retry behaviour when the file appears empty.
- grep -iq with tail — extracts and displays the last five matching lines for context.

Code:

```
#!/bin/bash
# ============================================================
# Script 3: Disk and Permission Auditor
# Course  : Open Source Software — VITyarthi
# Author  : Yash Tripathi | Roll No: 24BCE10603
# What it does: Loops through key system directories and reports
#               permissions, ownership, and size for each.
#               Also checks Git's own config file locations.
# ============================================================
# ANSI colours
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
```

```
CYAN='\033[0;36m'
BOLD='\033[1m'
NC='\033[0m'
# Command-line arguments
# $1 is the log file path; $2 is the keyword (default: "error")
LOGFILE=$1
KEYWORD=${2:-"error"}
# Counter and retry variables
COUNT=0
RETRY_LIMIT=3
RETRY_COUNT=0
echo -e "${BLUE}============================================================${NC}"
echo -e "${BOLD}        LOG FILE ANALYZER                          ${NC}"
echo -e "${BLUE}============================================================${NC}"
echo ""
# Validate that the user actually gave us a file path
if [ -z "$LOGFILE" ]; then
   echo -e "  ${RED}ERROR:${NC} No log file specified."
   echo -e "  Usage  : $0 /path/to/logfile [keyword]"
   echo -e "  Example: $0 /var/log/syslog error"
   exit 1
fi
# Check the file exists — [ -f ] tests for a regular file
if [ ! -f "$LOGFILE" ]; then
   echo -e "  ${RED}ERROR:${NC} '$LOGFILE' not found."
   echo -e "  Check the path and try again."
   exit 1
fi
echo -e "  ${CYAN}Log File :${NC} $LOGFILE"
echo -e "  ${CYAN}Keyword  :${NC} '${YELLOW}$KEYWORD${NC}'"
echo ""
echo -e "${BLUE}------------------------------------------------------------${NC}"
# Do-while style retry loop — Bash has no native do-while,
# so we use "while true" and break out once the file has content.
# This handles the edge case where a log file exists but is empty.
while true; do
   if [ -s "$LOGFILE" ]; then
      break   # File is non-empty — proceed
   fi
   RETRY_COUNT=$((RETRY_COUNT + 1))
   if [ $RETRY_COUNT -ge $RETRY_LIMIT ]; then
      echo -e "  ${YELLOW}WARNING:${NC} File appears empty after $RETRY_LIMIT checks."
      echo -e "  Continuing — result will show 0 matches."
      break
   fi
   echo -e "  File looks empty. Retrying ($RETRY_COUNT/$RETRY_LIMIT)..."
   sleep 1
done
echo -e "  Scanning for keyword '${YELLOW}$KEYWORD${NC}'..."
echo ""
# while IFS= read -r reads the log file one line at a time.
# IFS= prevents leading/trailing whitespace from being stripped.
# -r prevents backslashes from being interpreted as escape characters.
while IFS= read -r LINE; do
   # grep -iq does a case-insensitive quiet match on each line
   if echo "$LINE" | grep -iq "$KEYWORD"; then
      COUNT=$((COUNT + 1))   # arithmetic expansion increments the counter
```

```
 fi
done < "$LOGFILE"
# Print the summary
echo -e "${BLUE}------------------------------------------------------------${NC}"
echo -e "  ${GREEN}SCAN SUMMARY${NC}"
echo -e "${BLUE}------------------------------------------------------------${NC}"
if [ $COUNT -gt 0 ]; then
   echo -e "  Keyword '${YELLOW}$KEYWORD${NC}' found ${GREEN}${BOLD}$COUNT${NC} time(s) in:"
   echo -e "  $LOGFILE"
   echo ""
   echo -e "  ${CYAN}Last 5 matching lines:${NC}"
   echo -e "${BLUE}------------------------------------------------------------${NC}"
   # grep + tail extracts the last 5 lines containing the keyword
   grep -i "$KEYWORD" "$LOGFILE" | tail -5 | while IFS= read -r MATCH; do
      echo -e "  ${YELLOW}>>>${NC} $MATCH"
   done
else
   echo -e "  Keyword '${YELLOW}$KEYWORD${NC}' was ${RED}not found${NC} in $LOGFILE."
   echo -e "  Try a different keyword: ${CYAN}warning, fail, denied, critical${NC}"
fi
echo ""
echo -e "${BLUE}============================================================${NC}"
```

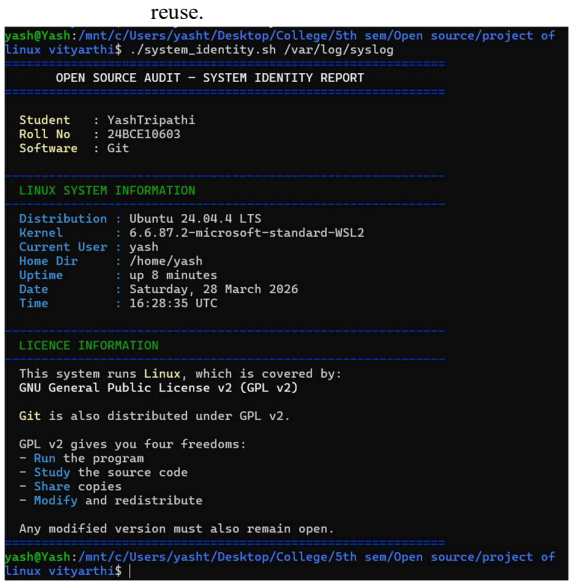
Script 5 — Open Source Manifesto Generator

Explaination:

This script asks the user three interactive questions about their open-source philosophy, collects system metadata, generates a formatted manifesto file using a heredoc, and saves it with a timestamped filename.

Concepts demonstrated

- **read command** — read -p prompts the user and stores input in variables like TOOL, FREEDOM, and BUILD.
- **Alias as function** — today() function mimics the behaviour of an alias, showing how to wrap simple commands for reuse.



- **Heredoc (<<EOF)** — writes multi-line text to a file cleanly without multiple echo statements; variables inside are expanded automatically.
- **Command substitution** — $(whoami), $(hostname), $(date +%Y%m%d) capture system information at runtime.
- **Filename generation** — manifesto_${USER_NAME}_$(date +%Y%m%d).txt creates a unique, traceable output file per user per day.
- **cat with redirection** — cat > "$OUTPUT" << EOF combines file creation and heredoc content writing.
- **ANSI colour codes** — \033[0;32m etc. add coloured terminal output for better user experience.

Code:

```
#!/bin/bash
# ============================================================
# Script 3: Disk and Permission Auditor
# Course  : Open Source Software — VITyarthi
# Author  : Yash Tripathi | Roll No: 24BCE10603
# What it does: Loops through key system directories and reports
#               permissions, ownership, and size for each.
#               Also checks Git's own config file locations.
# ============================================================
# ANSI colours
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
PURPLE='\033[0;35m'
CYAN='\033[0;36m'
BOLD='\033[1m'
NC='\033[0m'
# Alias concept demonstrated as a named function.
# In an interactive shell you'd write: alias today='date "+%d %B %Y"'
# Here we wrap the same logic in a function so the script can call it.
today() {
   date '+%d %B %Y'}
clear
echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════════${NC}"
echo -e "${BOLD}        OPEN SOURCE MANIFESTO GENERATOR                  ${NC}"
echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════════${NC}"
echo ""
echo -e "${YELLOW}This tool asks you three questions about your relationship${NC}"
echo -e "${YELLOW}with open-source software, then generates a personal      ${NC}"
echo -e "${YELLOW}philosophy statement and saves it to a file.              ${NC}"
echo ""
echo -e "Press ${CYAN}Enter${NC} to begin..."
read
clear
# ── Collect user input ──────────────────────────────────────────
echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════════${NC}"
echo -e "${GREEN}Question 1 of 3${NC}"
echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════════${NC}"
echo ""
echo -e "${YELLOW}Name one open-source tool you use every day:${NC}"
echo -e "(Examples: Git, Firefox, Linux, VLC, Python, LibreOffice)"
echo ""
# read -p prompts inline and stores the response in the variable
read -p "➜ " TOOL
echo ""
echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════════${NC}"
echo -e "${GREEN}Question 2 of 3${NC}"
echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════════${NC}"
echo ""
echo -e "${YELLOW}In one word, what does 'freedom' mean to you in software?${NC}"
echo -e "(Examples: choice, transparency, control, privacy, community)"
echo ""
read -p "➜ " FREEDOM
echo ""
echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════════${NC}"
echo -e "${GREEN}Question 3 of 3${NC}"
echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════════${NC}"
echo ""
echo -e "${YELLOW}Name one thing you would build and share freely:${NC}"
```

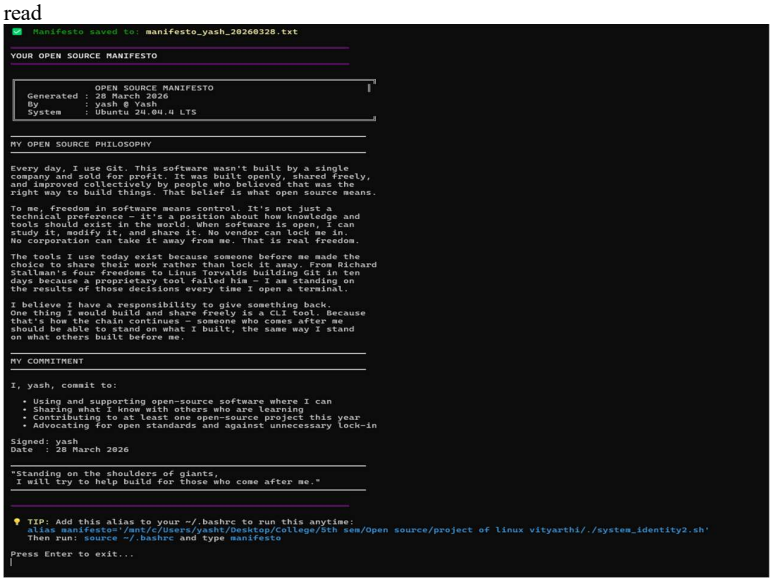
```
echo -e "(Examples: a learning platform, a document editor, a CLI tool)"
echo ""
read -p "➜ " BUILD
echo ""
# ── Gather metadata ──────────────────────────────────────────────────────
DATE=$(today)          # uses our alias-style function
USER_NAME=$(whoami)
HOSTNAME=$(hostname)
DISTRO=$(grep -w "PRETTY_NAME" /etc/os-release 2>/dev/null \
        | cut -d= -f2 | tr -d '"' \
        || echo "Linux")
# Output filename includes the username and today's date
OUTPUT="manifesto_${USER_NAME}_$(date +%Y%m%d).txt"
# ── Write manifesto using heredoc (<<EOF) ────────────────────────────────
# The heredoc writes everything between EOF markers to the file
# in one clean block — much neater than a chain of echo >> lines.
# Variables inside heredoc are still expanded automatically.
cat > "$OUTPUT" << EOF
# ── Display the result ───────────────────────────────────────────────────

clear
echo -e "${GREEN}✅ Manifesto saved to: ${YELLOW}$OUTPUT${NC}"
echo ""
echo -e
"${PURPLE}═══════════════════════════════════════════════════════════════════════${NC}"
echo -e "${BOLD}YOUR OPEN SOURCE MANIFESTO${NC}"
echo -e
"${PURPLE}═══════════════════════════════════════════════════════════════════════${NC}"
echo ""

# cat displays the saved file back to the screen
cat "$OUTPUT"

echo ""
echo -e
"${PURPLE}═══════════════════════════════════════════════════════════════════════${NC}"
echo ""

# Alias tip — shows the user how to make this a quick command
echo -e "${YELLOW}💡 TIP:${NC} Add this alias to your ~/.bashrc to run this anytime:"
echo -e "   ${CYAN}alias manifesto='$PWD/$0'${NC}"
echo -e "   Then run: ${CYAN}source ~/.bashrc${NC} and type ${CYAN}manifesto${NC}"
echo ""
echo -e "Press Enter to exit..."
read
```



Conclusion

This audit of Git goes beyond just understanding it as a version control tool — it reveals how deeply its design, licence, and philosophy are connected to the broader idea of open source. Git was not created in isolation; it was a direct response to a real-world problem involving dependency on proprietary software. That origin is reflected in its distributed nature, which removes central control and ensures that no single entity can restrict access or ownership again.

From a technical perspective, Git demonstrates how efficient design — minimal dependencies, strong integration with Linux systems, and powerful branching capabilities — can scale from individual developers to massive global projects. Its presence across modern development workflows, from local repositories to CI/CD pipelines, shows that it is not just a tool but a foundational layer of today's software ecosystem.

The GPL v2 licence plays a critical role in preserving this openness. By enforcing copyleft, it ensures that the freedoms associated with Git are not lost over time. This creates a system where innovation can continue without the risk of being closed off, aligning with the core values of free software.

At the same time, this audit highlights that open source is not just a technical model but also an ethical one. Questions around contribution, fairness, and community responsibility remain important, especially as large organisations increasingly rely on open-source tools. Git stands as a strong example of how community-driven development, when supported by the right licence and governance model, can produce software that is both highly practical and philosophically meaningful.

In conclusion, Git represents more than just a successful software project — it represents a shift in how software is built, shared, and controlled. Its impact on modern development is undeniable, and its principles continue to shape the future of open-source innovation.

References

The following were used for research. Everything in this report is written in my own words.

- GNU Project. The Free Software Definition. gnu.org/philosophy/free-sw.html
- Open Source Initiative. The Open Source Definition. opensource.org/osd
- Raymond, E. S. The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar
- Torvalds, L. Just for Fun: The Story of an Accidental Revolutionary. HarperBusiness, 2001.
- Chacon, S. & Straub, B. Pro Git (2nd ed.). Apress, 2014. git-scm.com/book
- Shotts, W. The Linux Command Line. linuxcommand.org
- GNU Bash Manual. gnu.org/software/bash/manual
- SPDX License List — GPL-2.0-only. spdx.org/licenses/GPL-2.0-only.html
- Git official documentation. git-scm.com/docs
- Linux kernel mailing list archives. lore.kernel.org
- Linux man pages. man7.org/linux/man-pages